

Git Collaboration

Alexis Shakas Ian Wasser (TA)

ETH Zurich

2026-03-24

Today's Agenda

1. **From Local to Remote**
2. **Connecting to GitHub**
3. **Accessing a Repo**
4. **Teamwork**
5. **Pull Requests**
6. **GitHub Features**

Organizational

Lecture	Date	T.H.A.	Room
1: Kickoff	17/2	N	RZ D8
2: AI assistance and tools	24/2	N	RZ D8
3: Project-based teamwork	3/3	N	RZ D8
4: Intro to Git	10/3	N	RZ D8
5: Git continued	17/3	N	ML H41.1
6: Git advanced	24/3	N	RZ D8
7: AI-assisted Python	31/3	Y	ML H41.1
8: The 3 R's in Python	14/4	Y	ML H41.1
9: Python packaging ecosystem	21/4	Y	RZ D8
10: Testing and debugging	28/4	Y	RZ D8
11: Joker lecture	5/5	N	ML H41.1
12: OOP	12/5	Y	RZ D8
13: Project distribution	19/5	N	RZ D8
14: Project marketplace	26/5	N	RZ D8

T.H.A.: Take home assignment

Important project dates

Keep these dates in mind for your project planning!

Milestone	Date	TODO
Project chosen	13/3	Done
Project Structured	27/3	Submit (markdown) file with project structure
Implementing	31/3	8 weeks to code and refine project
Project draft	15/5	Pre-submission of project for feedback
Project submission	29/5	Final submission of project on Git (tag)

Recap

Lecture 4 — Git Basics

- Commits, snapshots, history
- `git init`, `git add`, `git commit`
- `git status`, `git log`, `git diff`
- `.gitignore` & commit best practices

Lecture 5 — Branches

- `git branch`, `git switch`, `git checkout`
- HEAD & detached HEAD state
- `git merge` & `git rebase`
- Resolving merge conflicts
- Git workflows

One thing is still missing: all of this lives only on **your machine**.

⇒ Today learn how to connect your repository to the world and start **collaborating**.

We will use **GitHub** as our remote hosting service — but the concepts apply to any Git hosting platform (GitLab, Bitbucket, ETH Gitlab etc.).

From Local to Remote

The Old Way: Patch Files

From Local to Remote

You already know that Git stores diffs between commits. You can also export them as patch files:

```
git diff commit_hash > update.patch  
git apply update.patch
```

In the early days of open source, contributors would email these patches to project maintainers. **Fun fact:** This is still how contributions reach Linus Torvalds for the Linux kernel **on GitHub**.

Git

Local to your computer

Enables **Version Control** of
your project



GitHub

Cloud Storage for your Git
project

Hosted by **Microsoft**



Connecting to GitHub

Ways to Authenticate with GitHub

Connecting to GitHub

There are several ways to connect your local Git to GitHub:

SSH Keys

Recommended

Cryptographic key pair — no password needed per interaction. Set up once, works forever.

```
git clone  
git@github.com:user/repo.git
```

HTTPS + Personal Access Token

Use a token instead of a password. Must store or re-enter the token.

```
git clone https://github.com/  
user/repo.git
```

GitHub CLI (gh)

Authenticates via browser login. Wraps Git commands with extra GitHub features.

```
gh auth login  
gh repo clone user/repo
```

HTTPS + Password

Deprecated

GitHub removed password-based HTTPS authentication in August 2021 for security reasons.

We will use **SSH keys** in this course — the most common method for developers.

SSH keys are a pair of files that prove your identity to a remote server without a password.

Fingerprint analogy: Your **private key** is your fingerprint — unique and always with you. The **public key** is a scan of that fingerprint, registered with GitHub's door lock. When you connect, the lock checks the print against its record. If it matches, you're in.

Private key — stays on your machine

Public key — copy this to GitHub

```
~/.ssh/id_ed25519
```

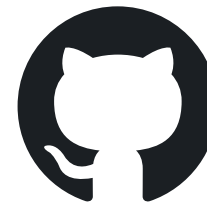
```
-----BEGIN OPENSSH PRIVATE KEY-----  
b3BlbnNzaC1rZXktdjEAAAAA...  
... (many lines) ...  
-----END OPENSSH PRIVATE KEY-----
```

Never share this file.

```
~/.ssh/id_ed25519.pub
```

```
ssh-ed25519 AAAAC3Nza...  
you@mail.com
```

One line. Safe to share. This is what you paste into GitHub under **Settings** → **SSH Keys**.



Copy to GitHub



Public



Public



Private



Note:

You should already have SSH keys set up from the earlier guide. If not, see the SSH Key Guide on Moodle.

P1: *Accessing a Repo*

Cloning a Repository

When you want to contribute to an existing repository, you **clone** it:

```
git clone git@github.com:username/repo-name.git
```

This downloads the full history and automatically sets up origin as the remote.

You can verify what remote repository you have connected to with:

```
git remote -v
```

and you should get something like:

```
origin  git@github.com:username/repo-name.git (fetch)  
origin  git@github.com:username/repo-name.git (push)
```

origin is just the default name Git gives to the remote you cloned from. You could rename it but everyone uses origin by convention.

1. Clone the class repository:

```
git clone git@github.com:shakasaki/a-purrfect-day.git  
cd a-purrfect-day
```

2. Open `story.md` and read the story's beginning
3. Confirm your remote is set up: `git remote -v`

Other people may push commits to the remote while you work locally. Your local repository does **not** update automatically. You can:

1. Get the changes without altering your local branch. Inspect the new commits and decide how to integrate them.

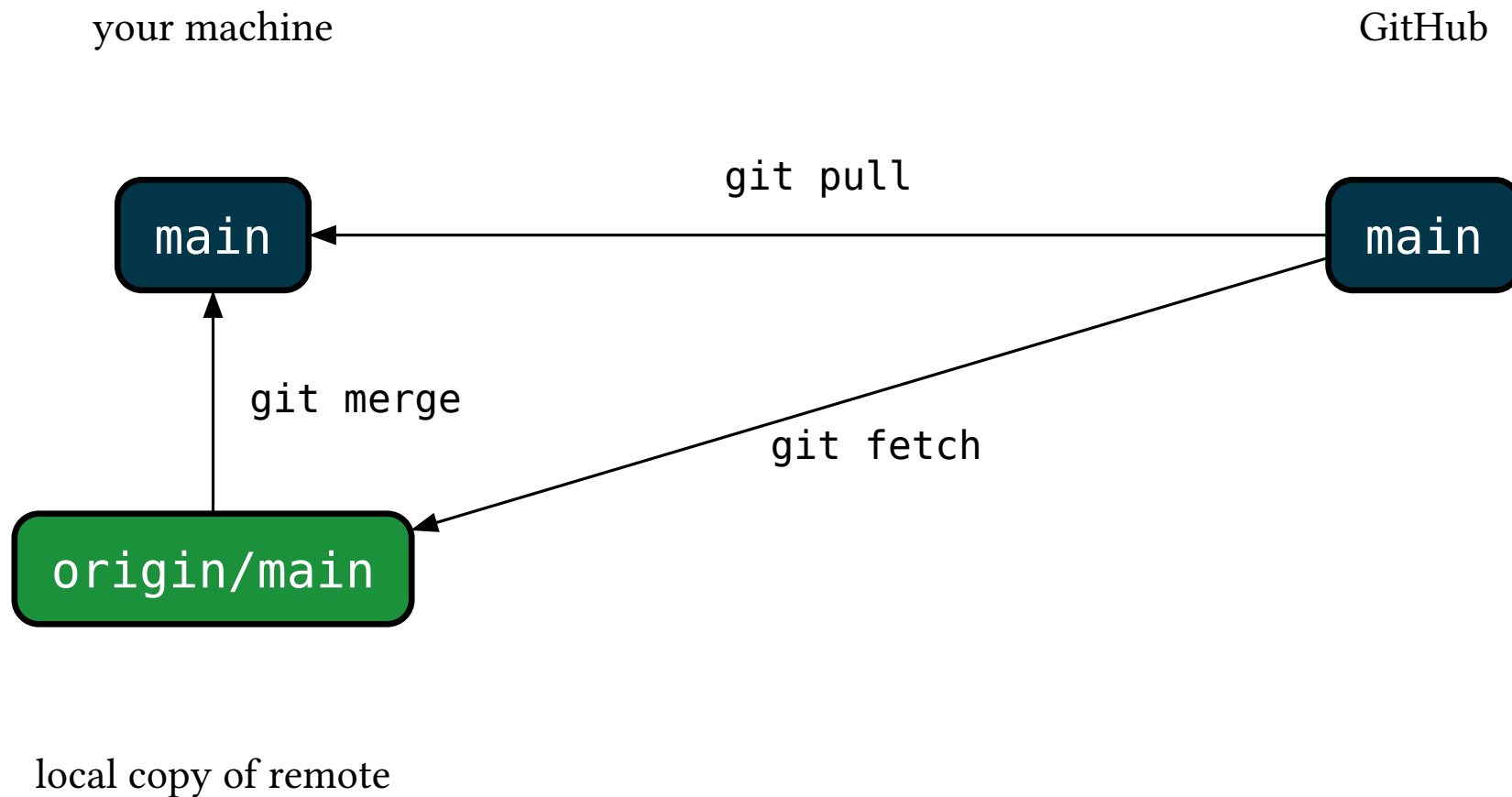
```
git fetch
```

2. Get the changes and let Git try to integrate them automatically.

```
git pull    # equivalent to: git fetch && git merge origin/main
```

git fetch vs git pull — Diagram

P1: Accessing a Repo



Rebase vs Merge on Pull

In our first Git lecture we set up the command

```
git config --global pull.rebase true
```

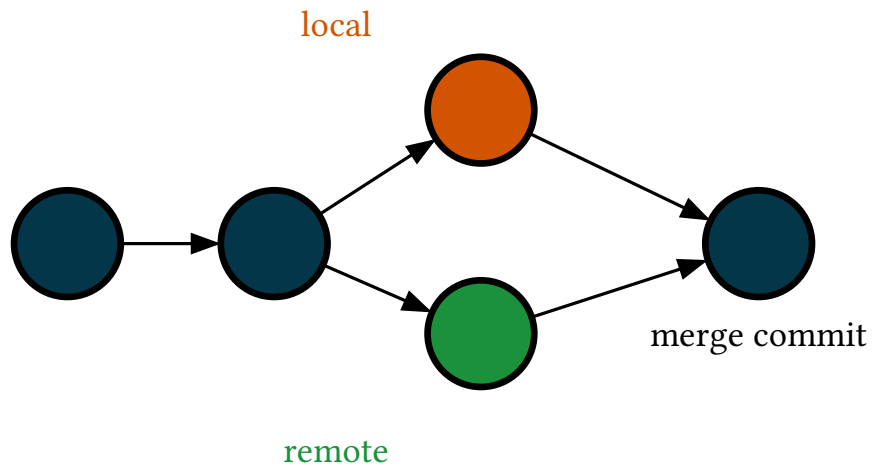
This basically tells Git to automatically rebase the changes that come in with `git pull` and to not create a new branch and merge. It looks cleaner in the history of your project.

Merge vs Rebase on Pull

`git pull (merge)`

or equivalently

`git fetch + git merge`

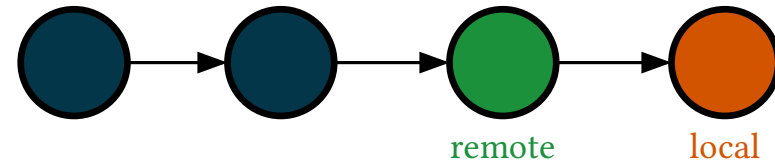


Creates a merge commit
— history branches and rejoins

`git pull --rebase (rebase)`

or equivalently

`git fetch + git rebase`



Replays your commit on top
— history stays linear

1. Fetch the latest changes from the remote:

```
git fetch
```

2. Inspect what arrived:

```
git log main..origin/main --oneline
```

3. Merge the remote changes into your branch with `git merge origin/main`
4. Confirm your branch is up to date with `git log`

Temporarily storing changes

Scenario: You're editing `story.md` when you remember a teammate just pushed new commits. You try `git pull` but Git refuses to pull with local changes that haven't been committed.

Solution: `git stash` temporarily shelves your uncommitted changes, leaving the working directory clean.

```
git stash      # save uncommitted changes and clean the working directory
git pull      # now the pull works cleanly
git stash pop  # restore your changes on top of the new commits
```

1. Open `story.md` and add a sentence. **Do not stage (add) or commit it**
2. Try to pull: `git pull`

Observe: Git refuses because you have local changes

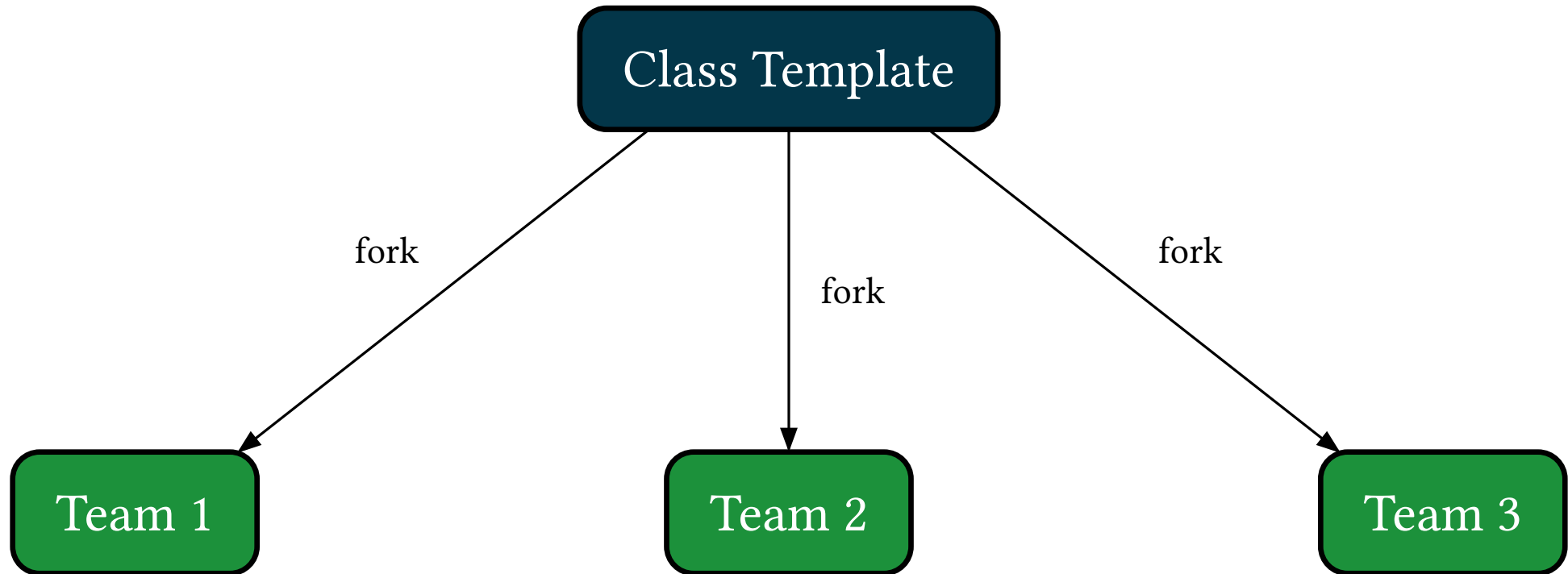
3. Stash your changes: `git stash` (Observe the file)
4. Pull again; it should work now
5. Restore your changes: `git stash pop`

Observe the file again: your sentence is back in `story.md`

P2: Your Team Repo

Forking

Forking creates your own copy of a repository under your GitHub account. Your team owns the fork to work independently on the project without affecting the original repository.



One person per team:

1. **Fork** (top right) the class template
2. In your fork: go to **Settings** → **Collaborators** → **Add people**
3. Add your teammates by their GitHub username

Everyone else:

5. Accept the collaborator invite (check your GitHub or email)
6. Clone your team's fork:
7. Confirm with `git remote -v` that the origin points to your team's fork

When multiple people push to the same branch, only the **first** push succeeds. Everyone else gets rejected:

```
! [rejected]  main -> main (fetch first)
error: failed to push some refs
hint: Updates were rejected because the remote contains work
hint: that you do not have locally.
```

This is **not** an error you caused — it means the remote has moved ahead of you. The fix is to pull the new commits first, then push:

```
git pull    # rebase your commit on top of the new ones
git push    # now it works
```

1. Open `story.md` and add **one sentence** to the story
2. Stage and commit the changes (`git add`, `git commit`)
3. Push your changes to remote with `git push`.

If your push was rejected:

4. Run `git pull` — you may see a conflict in `story.md`.
5. Open file / resolve the conflict (keep both sentences) / save
6. Run `git add .` and `git rebase --continue`
7. Push again: `git push`

At the end, everyone's sentence should be in `story.md` on your repo!

Now that multiple people have contributed to `story.md`, a natural question arises: **who wrote which line?**

```
git blame story.md
```

The output shows each line of the file annotated with the commit hash, author name, and date of the last change:

```
^3f2a1bc (Alice 2026-03-24 10:12:05) Once upon a time, there was a cat.  
d4e8f901 (Bob 2026-03-24 10:15:42) The cat loved to sleep in the sun.  
a1b2c3d4 (Carlos 2026-03-24 10:18:30) One day, the cat discovered coffee.
```



Note:

`git blame` is useful when investigating a bug — you can immediately see who introduced a specific line and look up that commit with `git show <hash>`.

Pull Requests

Why branches and Pull Requests?

Scenario: Your team is working on the project. Everyone pushes directly to main. Bob pushes a bug that breaks the code. Alice pulls the broken code without realising. Carlos overwrites Bob's fix with an older version. Nobody knows who changed what or why.

Why branches and Pull Requests?

Scenario: Your team is working on the project. Everyone pushes directly to main. Bob pushes a bug that breaks the code. Alice pulls the broken code without realising. Carlos overwrites Bob's fix with an older version. Nobody knows who changed what or why.

Solution: Each person works on their own **branch**. When a feature is ready, they open a **Pull Request** — a formal proposal to merge that branch into main. Before the merge happens, teammates review the code, leave comments, and discuss. Only reviewed code reaches main.

Pull Requests (PR) are a way to formally propose merging one branch into another. Before the merge happens, teammates review, leave comments, and discuss.

Without PRs

- Push directly to main
- No review/mistakes slip through
- Hard to track what changed and why

With PRs

- Changes are reviewed before merging
- Discussion is visible to the whole team

GitHub calls them **Pull Requests**. GitLab calls them **Merge Requests**. They are the same concept.

1. Create a new branch: `git switch -c your-name`
2. Add a short paragraph to `story.md` and add/commit
3. Push your local branch to the remote repository:

```
git push --set-upstream origin your-name
```

4. Go to GitHub — you should see a prompt to open a PR
5. Open the PR and write a short description
6. Review **one** teammate's PR and comment before merging it
7. Once all PRs are merged, everyone pulls main: `git pull`

Observe the full story is assembled from all your contributions!

Tagging Releases

A **tag** is a permanent, named label on a specific commit. Unlike branches, tags never move.

Tags are used to mark **releases** or important milestones in your project.

```
git tag -a v1.0 -m "First release"  # tag the current commit
git push --tags                    # push all tags to GitHub
git tag                            # list all tags
```

Above, v1.0 is the **tag name** which is an arbitrary label you choose. The -m flag allows you to add a message describing the tag.

Note: Your **project submission** will have a tag called submission!

Once all contributions are merged into main, one person from the team:

1. Ensures they are on main and up to date: `git pull`
2. Create an annotated tag, for example:

```
git tag -a meow_v1.0 -m "The complete story"
```

3. Push the tag to GitHub:

```
git push --tags
```

4. Go on Github and find your tag under **Code** → **Tags**

Don't be too pushy

On the internet you might come across this command:

```
git push --force
```

This command is **destructive**. It **overwrites the remote history** with your local history — discarding any commits on the remote that you don't have locally.

Use it only when you know exactly what you are overwriting and why.

```
git push --force-with-lease
```

A safer alternative to --force. Before overwriting, it checks whether anyone else has pushed to the remote since you last fetched. If they have, it refuses.

Still **destructive** and still **dangerous** — but at least it won't silently destroy a teammate's work.

GitHub Features

GitHub provides a set of tools built around the repository:

- **Issues** — track bugs, tasks, and feature requests
- **Projects (Boards)** — Kanban-style task management
- **Wiki** — documentation hosted alongside your code
- **Pull Requests** — code review and discussion
- **Releases / Tags** — mark and distribute versions of your project
- **Insights** — visualise contribution history and activity

These are the same tools used by professional open source projects.

Take a few minutes to explore your team's repository on GitHub:

- Create an **Issue** describing something you'd like to add to your story. Assign the issue to a teammate
- Look at the **commit history** — can you see everyone's contributions?
- Navigate to the **Pull Requests** tab and read through the discussion on a PR
- Check out the **Wiki** tab — you can use this to write documentation for your project later on!

Recap

After this lecture, you should be able to:

- **Clone** a repository and understand what origin is
- **Explain** the difference between `git fetch` and `git pull`
- **Use** `git stash` and `git stash pop` to handle interrupted work
- **Resolve** a non-fast-forward push rejection
- **Open** a Pull Request and review a teammate's code
- **Fork** a repository and set up a shared team repository
- **Tag** a release with `git tag` and push it to GitHub
- **Inspect** authorship line-by-line with `git blame`